

Vista Food App

DFD Level-2 & Database Schema Diagram

Software Engineering Project | Bangladesh University of Professionals

Group Members

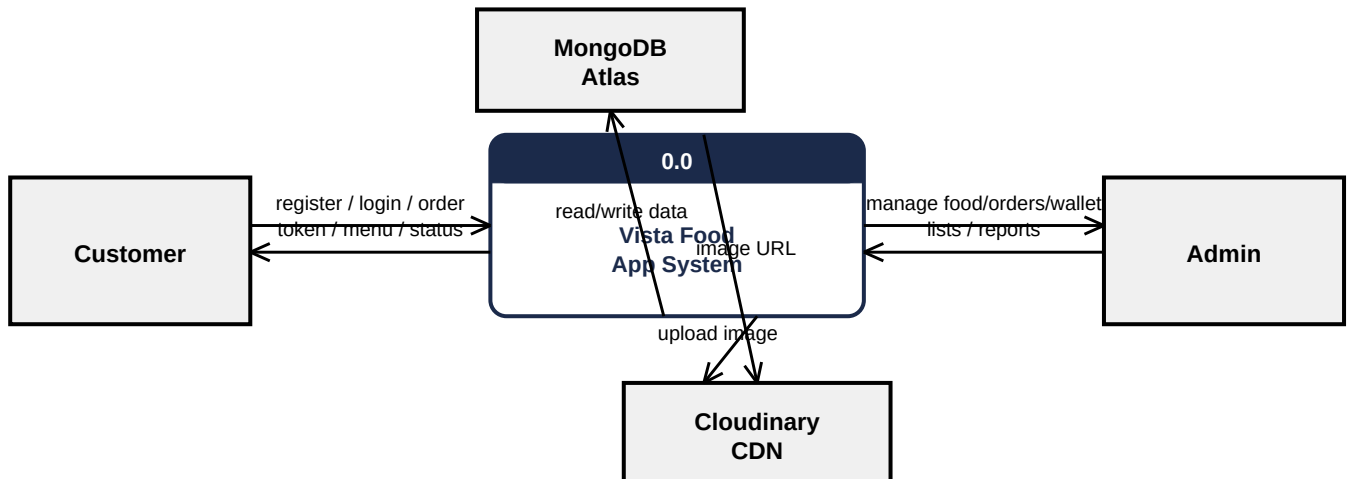
• Abrar Zahin	23524202100
• Abid Rejwan	23524202148
• Abdullah Bin Towhid	23524202054
• Tabida Ahmed	23524202134

April 2026

Vista Food App — Data Flow Diagrams

DFDs are drawn using the Gane & Sarson notation: external entities as plain rectangles, processes as rounded rectangles with a numbered top section, and data stores as open-ended horizontal bars. Arrows represent data flows and are labelled.

Level-0: Context Diagram

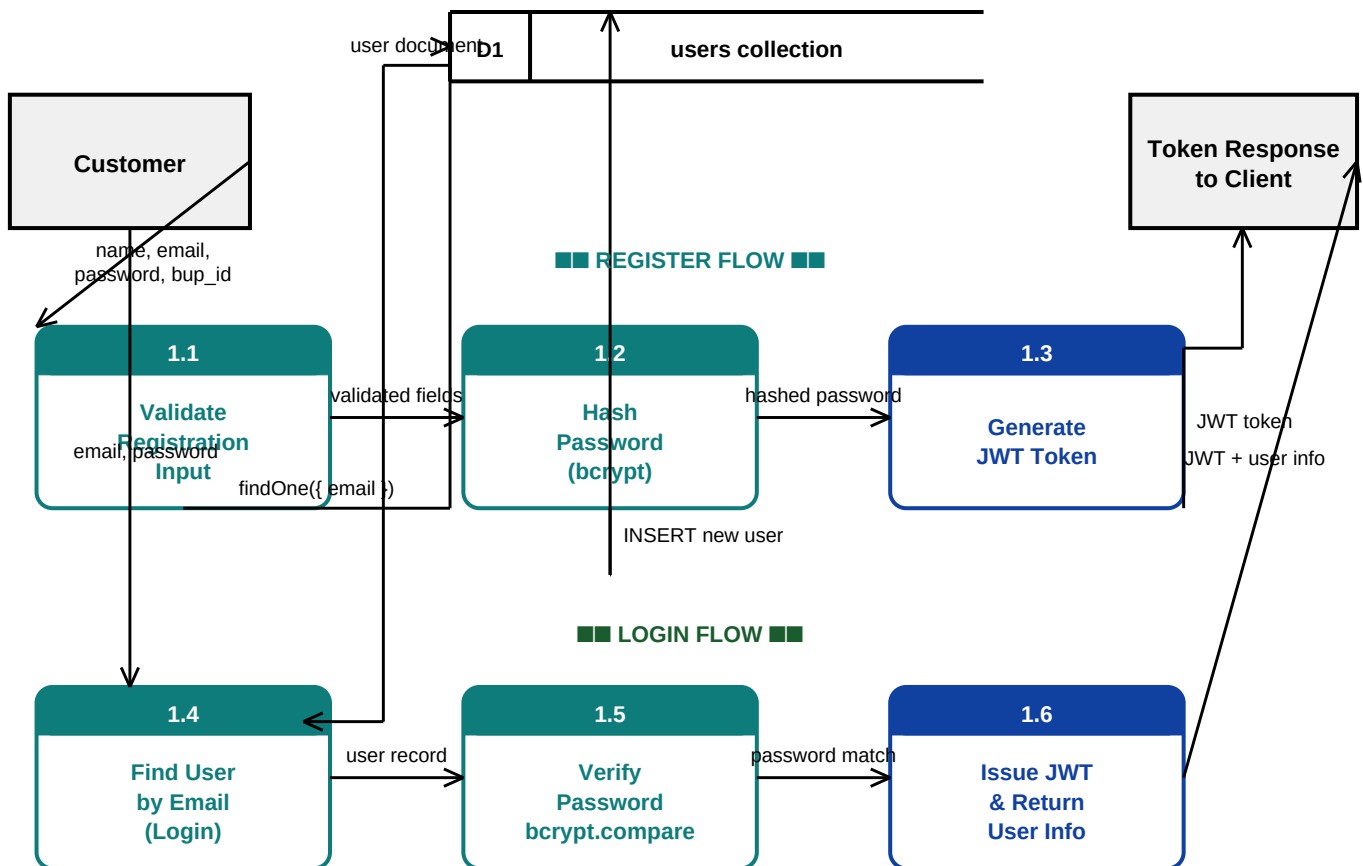


Level-1: Process Summary

ID	Process Name	Source File	Main Inputs	Main Outputs
1.0	User Management	userController.js	name, email, password, bup_id	JWT token → users DB
2.0	Food Management	foodController.js	food details + image file	food record → foods DB
3.0	Cart Management	cartController.js	userId, itemId, qty	cartData → users.cartData
4.0	Order Processing	orderController.js	userId, items[], amount	order + wallet update → orders DB
5.0	Wallet Management	adminRoute.js	bup_id, amount, operation	transaction → walletTx DB

Level-2: P1 — User Management

Source: `UserController.js` | Routes: `POST /api/user/register` `POST /api/user/login`



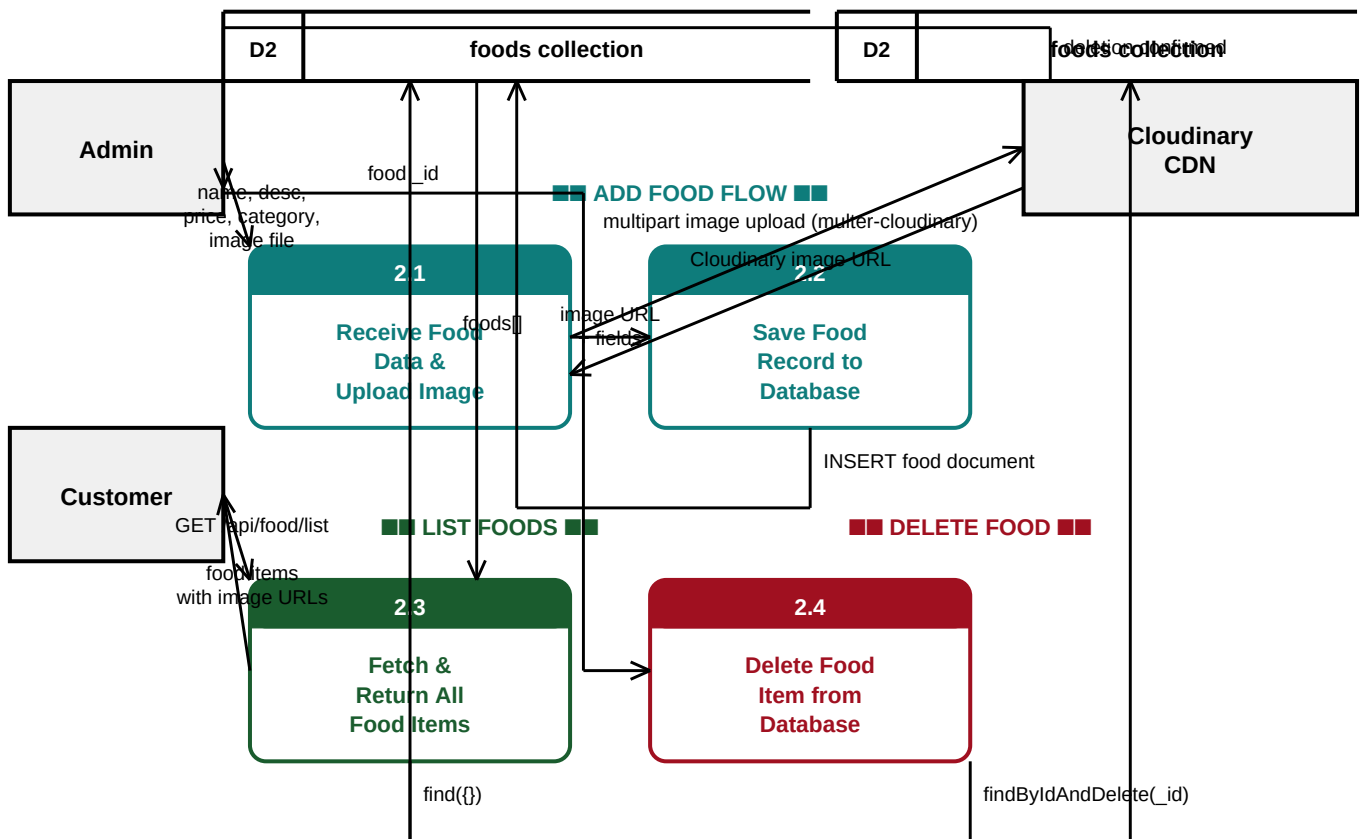
auth.js Middleware — applied to `/cart` and `/order` routes

Reads Authorization header → `jwt.verify(token, JWT_SECRET)`

Attaches `userId` to `req.body.userId` | Rejects invalid tokens immediately

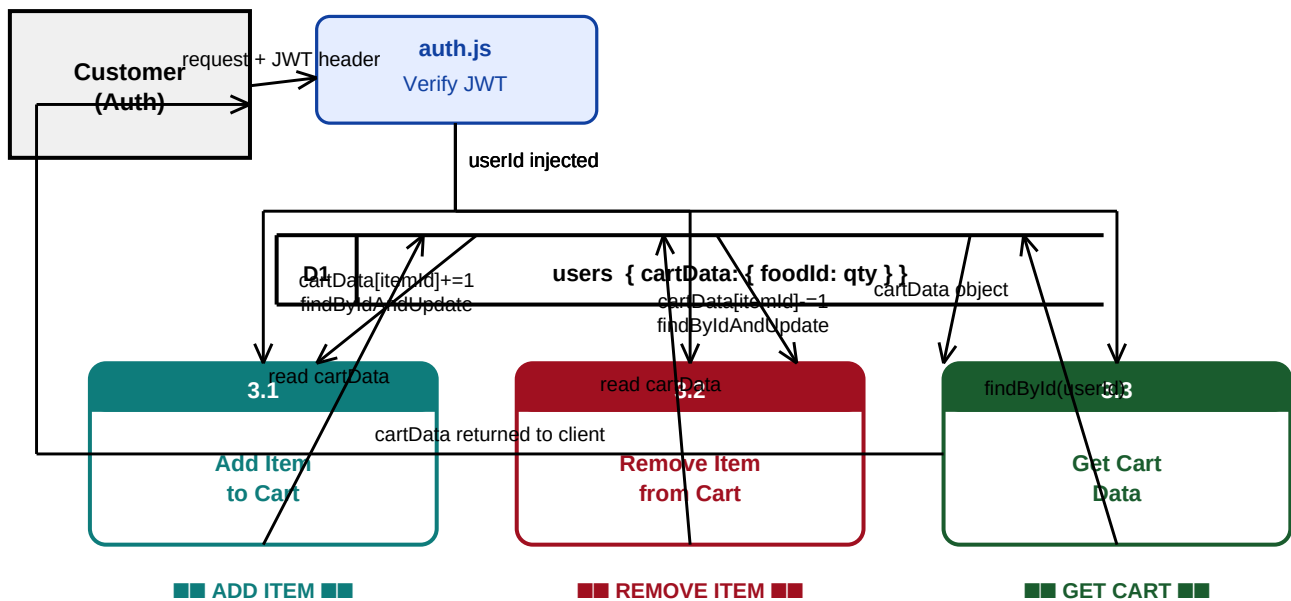
Level-2: P2 — Food Management

Source: foodController.js | Routes: POST /api/food/add GET /api/food/list POST /api/food/remove



Level-2: P3 — Cart Management

Source: cartController.js | Routes (all JWT-protected): POST /api/cart/add POST /api/cart/remove POST /api/cart/get

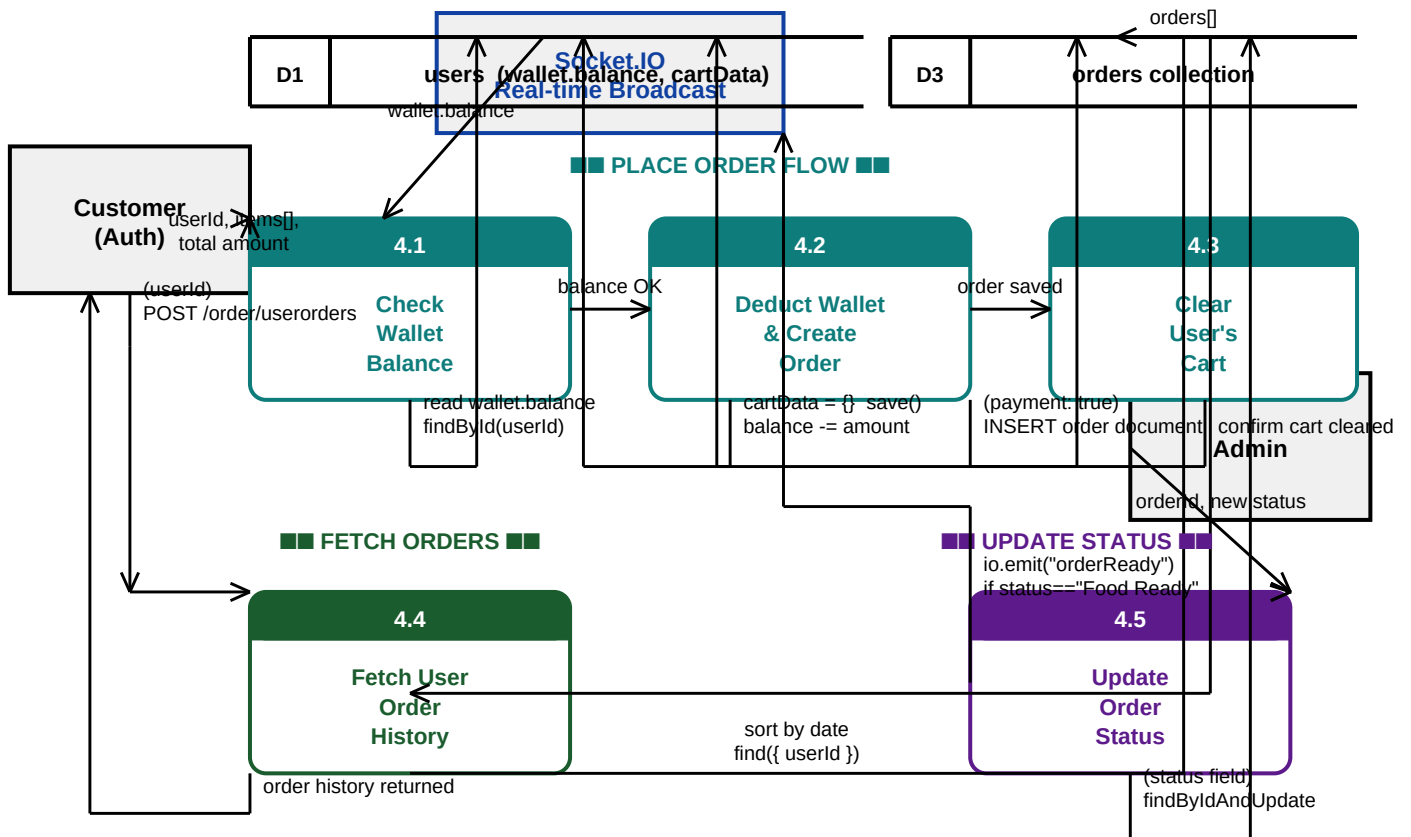


cartData is stored as an embedded Object inside the users document — no separate cart collection.

It is cleared to `{}` automatically after a successful order is placed (in Process 4.2).

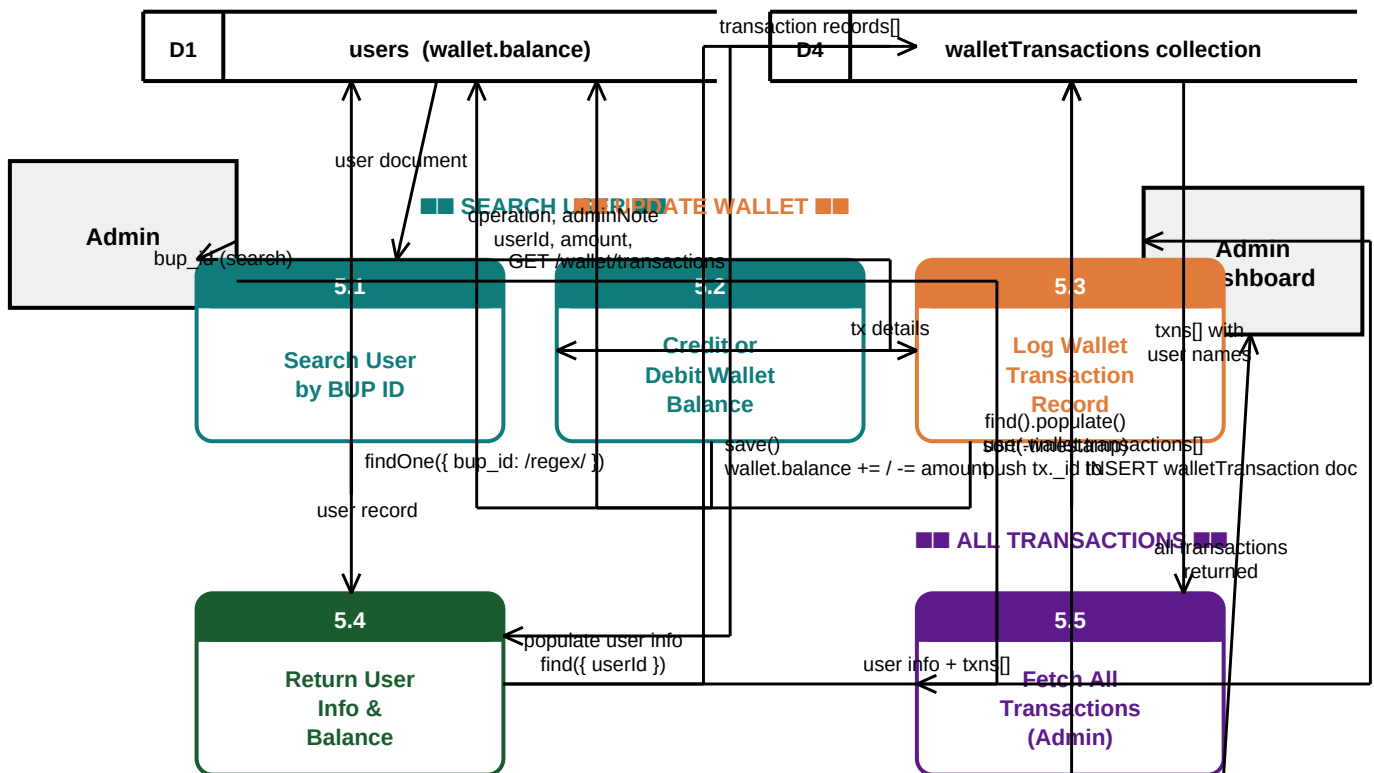
Level-2: P4 — Order Processing

Source: orderController.js | Routes: POST /api/order/place POST /api/order/userorders GET /api/order/list POST /api/order/status

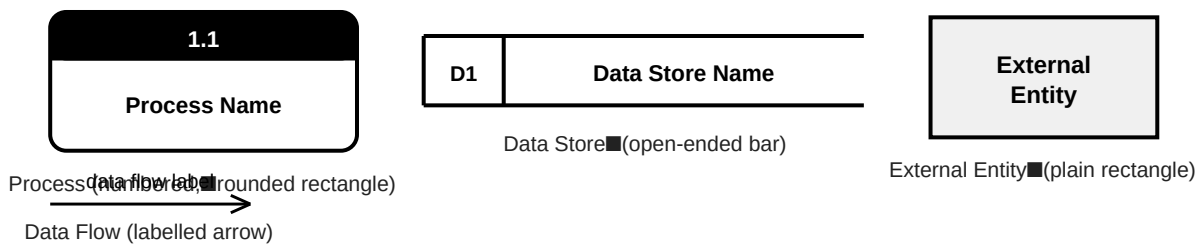


Level-2: P5 — Wallet Management

Source: adminRoute.js | Routes: GET /api/admin/wallet/user/:bupId POST /api/admin/wallet/update GET /api/admin/wallet/transactions



DFD Notation — Gane & Sarson Legend



3. Database Schema

Vista Food App uses MongoDB with Mongoose ODM. The four collections below are derived directly from the model source files. PK = Primary Key FK = Foreign Key (ObjectId reference).

Collection: users	Type	Constraints	Description
<u>_id</u>	ObjectId	PK, auto-generated	MongoDB primary key
name	String	required	Full name
email	String	required, unique	Login email
password	String	required	bcrypt hash (salt=10)
bup_id	String	required, unique, /^\d{11}\$/	BUP 11-digit ID
wallet.balance	Number	default: 0.0	Current balance in BDT
wallet.transactions	[ObjectId]	FK → walletTransactions._id	Array of transaction refs
cartData	Object	default: {}	{ foodId: quantity } — live cart

Collection: foods	Type	Constraints	Description
<u>_id</u>	ObjectId	PK, auto-generated	Primary key
name	String	required	Display name
description	String	required	Menu description
price	Number	required	Price in BDT
image	String	required	Cloudinary URL
category	String	required	e.g. Salad, Rolls, Desserts

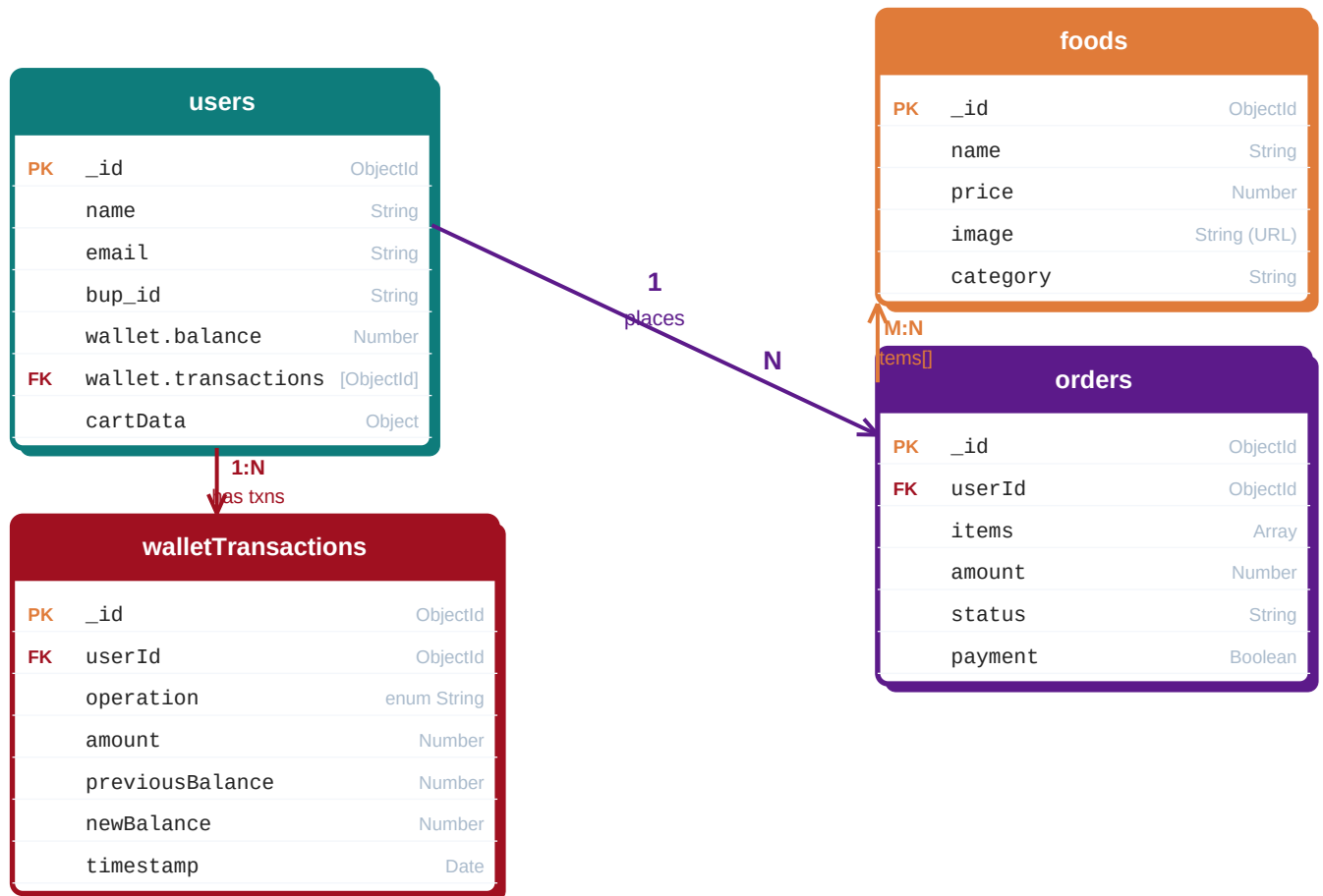
Collection: orders	Type	Constraints	Description
<u>_id</u>	ObjectId	PK, auto-generated	Primary key
userId	ObjectId	FK → users._id, required	Who placed the order
items	Array	required	Snapshots [{_id, name, price, quantity}]
amount	Number	required	Total in BDT
status	String	default: "Food Processing"	Processing / Preparing / Food Ready / Delivered

payment	Boolean	default: false	True once wallet deducted
data	Date	default: Date.now	Order timestamp

Collection: walletTransactions	Type	Constraints	Description
_id	ObjectId	PK, auto-generated	Primary key
userId	ObjectId	FK → users._id, required	Affected user
operation	String	enum: increase decrease	Credit or debit
amount	Number	required	Transaction amount in BDT
previousBalance	Number	required	Balance before transaction
newBalance	Number	required	Balance after transaction
adminNote	String	optional	Admin-provided reason
timestamp	Date	default: Date.now	When transaction occurred

4. Entity Relationship Diagram (ERD)

The ERD below shows all four MongoDB collections and their ObjectId-based relationships as implemented in the Mongoose schemas.



PK = Primary Key | FK = Foreign Key (ObjectId reference) | Arrows show referential relationships

Relationship Summary

From	To	Cardinality	How / Notes
users	orders	1 : N	orders.userId → users._id
users	walletTransactions	1 : N	walletTransaction.userId → users._id
users	walletTransactions	1 : N (embedded)	users.wallet.transactions[] stores tx IDs (populated with .populate())
orders	foods	M : N (denorm)	orders.items[] stores snapshots {_id, name, price, qty} at time of order